

From Skill 1.0 to Skill 2.0: A Modular Skill Middleware for Research Applications

Evidence from Fixed Income

Ye Qing

August 2026

Abstract

: The first generation of Skills was delivered in the form of "one directory holding an entire capability": globally available, loaded wholesale, and encapsulated and reused only as a whole Skill, with internal structure and dependencies opaque to the outside. As the thematic databases expanded to more than a dozen and the analytical perspectives accumulated to several hundred, the rising cost of wholesale loading and the excessive coarseness of the reuse granularity began to constrain production operations. This paper proposes Skill 2.0, which reframes the Skill from a "globally usable black box" into a "modular middleware": capabilities are decomposed into six module categories — SKILL.md, knowledge, templates, data_contract, scripts, and config — each of which carries an independent IN/OUT/DEP interface contract and can be invoked, tested, and versioned on its own. The middleware makes each of the five layers — data source, data access, knowledge, compute, and output — independently replaceable, decouples data-source bindings through the data_contract, and enforces a "framework before numbers" principle through the knowledge-layer validation baselines. Experience from a production fixed-income research system (13 thematic databases, roughly 500 analytical logics, and several architecture manuals in production) shows that the modular design significantly lowers integration cost and refines reuse granularity, and it carries the potential to transfer to equity, macro, and quantitative research.

Keywords: AI Agent, Skill Modularization, Middleware, Research Digitization, Fixed Income

1. Introduction

1.1 The Problem

Skill 1.0 answered the question of whether research capabilities could be encapsulated: the entire chain of pulling data, computing indicators, comparing thresholds, writing judgments, and producing reports was turned into Skills one by one and operated successfully in production. Yet as the scale of operations grew, the engineering limitations of first-generation Skills became increasingly apparent, converging on four "excesses":

- **Overly broad applicability:** A Skill is globally available by default, so any task may touch it. Each activation brings in content irrelevant to the current task, producing persistent context noise.
- **Overly coarse loading granularity:** The unit of execution is the entire Skill directory. Even when only the formula of a single perspective is needed, the full set of knowledge, templates, and scripts must be loaded.
- **Overly opaque internal structure:** Externally, only the high-level description in SKILL.md is visible; internal module dependencies, data sources, and calculation conventions cannot be inspected, so integrators can only borrow the Skill as a whole and cannot review it locally.
- **Overly large reuse units:** Reusing one perspective (for example, "the percentile judgment of overnight funding rates") under Skill 1.0 typically means copying an entire database Skill, and cross-database reuse produces large amounts of duplication and version drift.

These four limitations share a single root cause: **both the granularity of encapsulation and the unit of execution are "the whole Skill," which does not match the reality that a single task usually needs only a small piece of the capability.**

1.2 The Approach

The approach is decomposition: the smallest reusable unit of research capability should be "one module," not "the whole Skill." Skill 2.0 is a modular middleware — internally, it decomposes a Skill into modules with single responsibilities and independent contracts; externally, it assembles them on demand, so that taking a single module suffices for independent work.

"Middleware" here compresses two layers of meaning. First, every layer of the research chain (data source → data access → knowledge → compute → output) can exist and be replaced independently, and a change in the implementation of any layer does not force coordinated changes elsewhere. Second, capabilities are deposited as intermediate artifacts — structured products ready for assembly — and research ideas are combinations of these basic modules. This stance continues the argument of the v1 paper that "research capabilities are decomposed layer by layer and reassembled when needed"; what differs is that Skill 2.0 pushes the granularity of decomposition down to the **module** level and assigns each module a machine-verifiable interface contract.

2. From Skill 1.0 to Skill 2.0

2.1 The Form and Limitations of Skill 1.0

A typical Skill 1.0 is a flat directory: one SKILL.md plus a number of loosely organized documents and scripts, which enter the AI workflow through "global registration, semantic-match activation." Its limitations in production operations are summarized in the table below:

Dimension	Skill 1.0 form	Limitation in production
Scope of availability	Globally registered, activated automatically by task semantics	Unrelated tasks are also touched; activation noise rises
Loading granularity	The entire Skill directory is loaded as a whole	Getting one corner requires loading everything; high context overhead
Structural contract	No unified module specification; scripts and documents mixed	Internal dependencies invisible; integration only by borrowing the whole
Reuse unit	The entire Skill	Granularity too coarse; cross-database reuse relies on copy and trim
Data binding	Data sources hard-coded inside scripts	Switching data vendors requires modifying the data-access code

These five limitations are not a failure of the encapsulation idea itself but the outcome of the chosen encapsulation granularity. Skill 1.0 answered whether capabilities can be encapsulated; Skill 2.0 asks at what granularity and under what contract encapsulation is better suited to production.

2.2 The Skill 2.0 Design: Six Module Categories

Skill 2.0 prescribes that the standard structure of a Skill consists of six module categories:

```
skill-XXX/
├─ SKILL.md          <-- activation and routing (trigger conditions + perspective
list + module index)
├─ knowledge/        <-- knowledge module (formulas + thresholds + cases +
validation baselines)
├─ templates/        <-- output template module (table / chart / text forms)
├─ data_contract/    <-- data contract module (field definitions, decoupled from
data sources)
├─ scripts/          <-- executable script module (data-access engine / compute
engine, self-contained)
└─ config/           <-- configuration module (parameters / routing / version)
```

The responsibilities of the six module categories are:

- **SKILL.md**: activation instructions and routing table. It states which modules a Skill provides and under which conditions each is activated; it is the entry-point index for external callers.
- **knowledge/**: domain knowledge. It holds formulas, thresholds, cases, and validation baselines, and is the main surface in this system on which "the more it is used, the deeper it grows."
- **templates/**: output templates. They define the output forms of tables, charts, and text, decoupled from the computed results so that switching forms only requires switching templates.

- **data_contract/**: data contract. It declares only "which fields are needed, what types, what frequency," not "which API to pull from"; it is the source of data-source independence.
- **scripts/**: executable logic. Data-access engines, compute engines, and the like are required to be self-contained — shipped with dependency notes and a self-check script, and runnable independently of the Skill as a whole.
- **config/**: configuration. Parameters, data-source routing, and module versions let the same logic run under different parameter configurations.

Compared with Skill 1.0, the decisive change is not splitting into more directories but that **every module carries an independent responsibility and an independent interface contract**: a module can be invoked, tested, and versioned alone, and "the whole Skill" is no longer the only unit of execution.

2.3 Why Skills Need Not Be Globally Available

"Taking only one module" is the core usage of Skill 2.0 and the reason for dropping the default assumption that a Skill must be globally available. The argument proceeds at three levels:

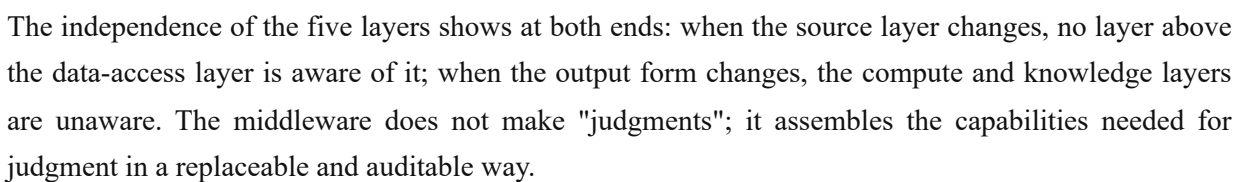
- **The cost trade-off of activation**: global availability means injecting the entire capability into the context of every task, yet the hit rate of this content in any given task is usually low. Once modularized, nothing is activated by default; the caller names and loads only the modules it needs by path — turning "inject everything" into "inject on demand."
- **Contract independence enables independent invocation**: a module has its own IN/OUT/DEP contract, so a caller only needs to satisfy its input declaration to run it, without understanding the rest of the Skill. The module becomes the smallest independently deliverable unit, whose paradigm is given in Section 3.3.
- **Cross-Skill composition becomes possible**: when the reuse unit drops from the Skill to the module, the cost of composing across databases falls sharply. For example, the judgment logic of the funding-percentile perspective can be referenced by several research databases at once without copying the entire funding database.

Thus Skill 2.0 does not require "global availability" but "availability of each module." Global registration degrades into one optional routing strategy rather than the only form.

3. Middleware Architecture and Interface Contracts

3.1 The Five-Layer Middleware

The middleware form of Skill 2.0 is a five-layer structure, in which every layer is independently replaceable:



Data-source independence is carried by the `data_contract`: the contract declares only "which fields I need," and at runtime the data-access layer performs the mapping to the concrete data source. For example, the `credit-spread` database maps its 11 analytical perspectives across implementations on 5 data sources that share a single data contract — `switch sources, keep perspectives`.

55

failures diagnosable at the contract level: whether the problem is a missing field, a wrong type, or a discontinued source can be checked item by item.

3.3 A Module IN/OUT/DEP Contract Example

Below is a contract-acceptance example from the live system — the "overnight funding price percentile" perspective of the funding database:

```
# data_contract/funding_pctile.yaml — module: funding, overnight price percentile
module: funding_pctile
version: 2.1

in:                                # Input declaration: fields + data-source
dependencies
  - dr001: {type: series, freq: day, aliases: [DR001, dr001_close]}
  - dr007: {type: series, freq: day, aliases: [DR007, dr007_close]}
  - start: {type: date}
  - window: {type: int, default: 250}

out:                                # Output declaration
  - pctile: {type: scalar, unit: pct}
  - regime: {type: label, enum: [easy, neutral, tight, very tight]}

dep:                                # Modules depended on at runtime
  - scripts/funding_fetcher.py      # data-access engine (self-contained)
  - knowledge/funding-factors.md    # thresholds and validation baselines
  - config/funding.yaml             # parameters and routing
```

The point of the acceptance example is independent invocation: a call initiated about the "overnight funding percentile" in a DASH conversation loads only the single `funding_fetcher` module and the dependencies declared in its `dep` clause; the code of the other 33 analytical perspectives of the funding database never enters the context. The caller can run and re-check against the contract without reading the whole database.

3.4 The Knowledge-Layer Validation Mechanism: Framework over Numbers

Beyond formulas and thresholds, the knowledge layer also holds **validation baselines** — a set of historical anchors denoting "plausible ranges" — used as a first-pass health check at the numeric level. Its operating principle is "framework over numbers":

- Validate the framework first: is the convention correct, is the data pipeline complete, is the computation executed per the contract;
- Then trust the numbers: once the framework is sound, if the value deviates from the validation baseline, suspect the data pipeline (missing data, wrong source, discontinued updates) rather than revising the conclusion first.

One real example: one week, the computed funding percentile fell in an extreme region above 90%; validation found that the upstream source was missing nearly three trading days of data, and the perspective returned to the 70% range once the data were completed. Without a validation baseline, the "extreme value" could easily have been written straight into a report as a market judgment. Framework correctness takes priority over single-value correctness — once the framework errs, no matter how precise the number, it is irreproducible and untraceable.

4. A Real Case

4.1 A Module-Level Invocation Example: DASH Takes a Single Module

The online fixed-income research system currently runs 13 thematic databases, roughly 500 analytical logics, and several architecture manuals (funding, credit spreads, convertibles, financial analysis, fund duration). At this scale, the returns of module-level invocation are direct.

Consider a funding-data pull in a DASH session as an example: the funding database has 34 analytical perspectives in total, yet the task invokes only its "data-access engine" module `scripts/funding_fetcher.py`; the other 33 perspectives are not loaded. Three changes result. First, context consumption and loading time fall in proportion. Second, the caller depends only on that module's `data_contract`, fully decoupled from the evolution of other perspectives. Third, the data-access engine can be reused independently by other sessions without "borrowing" the whole database. The same pattern applies equally to the credit-spread database (11 perspectives across 5 data sources) and the convertible-bond database (22 perspectives).

4.2 The Decomposition of the Technical Manuals

In the Skill 1.0 era, one manual played both roles — for users and for integrators — so "how to use" and "how it works internally" were mixed together in one place. In the Skill 2.0 rollout, this step was split: an intro manual for users answers "what it does and how to use it," while an architecture manual for integrators answers "how the modules are organized, what the interface contracts are, and how to integrate."

The significance of this split is that **the architecture manual is itself an instance of Skill 2.0**: the module diagram in the manual corresponds to the real directory structure, the interface contracts correspond to the `data_contract` and the modules' IN/OUT/DEP, and integrators can verify the manual against the real code item by item. Moreover, the charts in the manual are not schematic illustrations; they are high-fidelity reproductions drawn from DASH conversations, HTML manuals, and real data-run outputs, uniformly styled with the fioutput Navy design system — what the charts show is exactly what the production environment actually produced.

4.3 Switching Output Forms

For the same computation logic, the output can be a Word research report, an HTML dashboard, a WeChat public-account article, or a PPT. This goal already had a prototype in Skill 1.0 ("changing forms is like changing clothes"), but Skill 2.0 turns it into a mechanism: the compute layer produces one shared structured intermediate result, and the output layer selects the templates module and the rendering pipeline by target form — Word uses the research-report template, dashboards use the fioutput Navy HTML system, WeChat uses the official-account typesetting, and PPT uses image-based, bullet-oriented organization. When switching the output form, the compute and knowledge layers change not at all; the change is confined to the single replaceable surface of the output layer.

5. Discussion

5.1 Comparison with Skill 1.0 and Industry Solutions

Dimension	Skill 1.0	Skill 2.0 (modular middleware)	AutoGen / LangGraph	Traditional terminals (Bloomberg/Wind)
Reuse unit	The whole Skill	A single module	Agent / workflow	Function / formula
Loading granularity	Global loading	Load modules on demand	Execute by node	Query on demand
Interface contract	No unified specification	IN/OUT/DEP contract	Messages / graph edges	Proprietary API
Data-source decoupling	Hard-coded in scripts	data_contract abstraction	Must be orchestrated by the user	Terminal-bound
Domain knowledge accumulation	Documents and code mixed	Knowledge layer + validation baselines	Tool-chain oriented	Formula library

It should be clarified that frameworks such as AutoGen and LangGraph address the orchestration of agents and multi-agent collaboration; they carry no research-domain knowledge themselves and prescribe no encapsulation contract for domain capabilities. Skill 2.0 sits beneath them — first organizing domain capabilities under a modular contract, on top of which agent orchestration can be layered. The two are not substitutes.

5.2 Generality and Transfer

The abstraction of the five-layer middleware does not depend on fixed-income-specific business logic: the source, data-access, and output layers are largely reusable across domains as is, and the main new

efforts are in the knowledge and compute modules:

Target domain	Knowledge / compute module examples	Directly reusable surface
Equity research	Industry databases, factor pools for stock selection, financial-statement knowledge bases	Data-access layer, output layer
Macro research	Economic-indicator libraries, policy-event databases, forecasting-model libraries	Data-access layer, output layer
Quantitative strategies	Factor libraries, backtesting databases, risk-control rule libraries	Data-access layer, output layer

To be candid: only fixed income has run the full loop to date (13 thematic databases, roughly 500 analytical logics, several manuals in production); equity, macro, and quantitative research remain at the stage of design reasoning. The case for transfer rests on the abstract generality of the framework, not on complete empirical evidence.

5.3 Limitations

- The soundness of module contracts depends on the continued maintenance of the existing production system; building a new domain requires designing modules and contracts from scratch, a one-time cost;
- The self-containment principle of data-access engines brings a certain degree of dependency redundancy, and cross-module data consistency requires explicit validation at the middleware layer;
- The coverage depth of the knowledge-layer validation baselines depends on manual verification and accumulation, and it provides limited protection for perspectives not yet included in the baselines;
- The primary validation scenario remains fixed income, and cross-domain generalization has no complete empirical support.

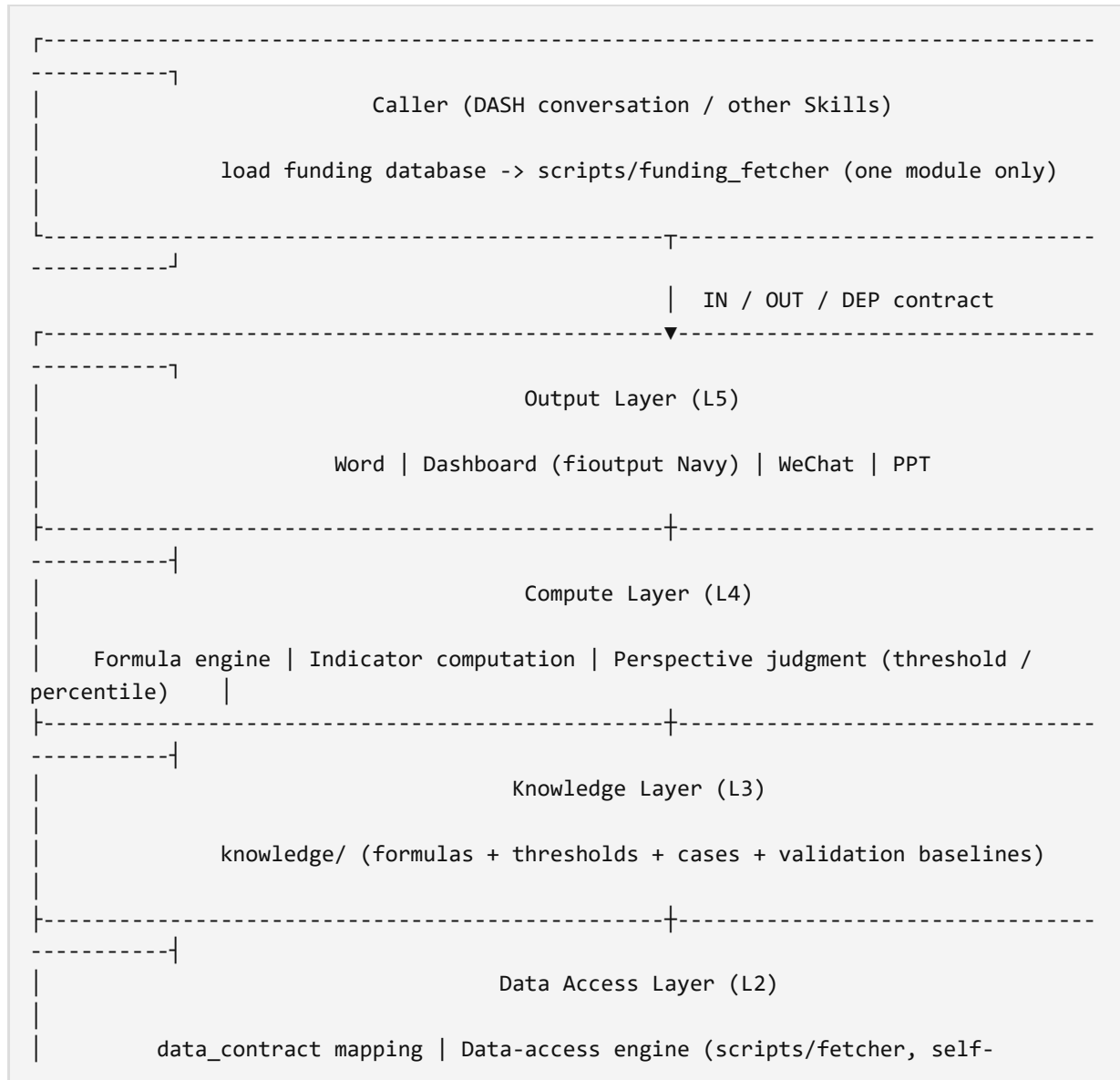
5.4 Sustained Value

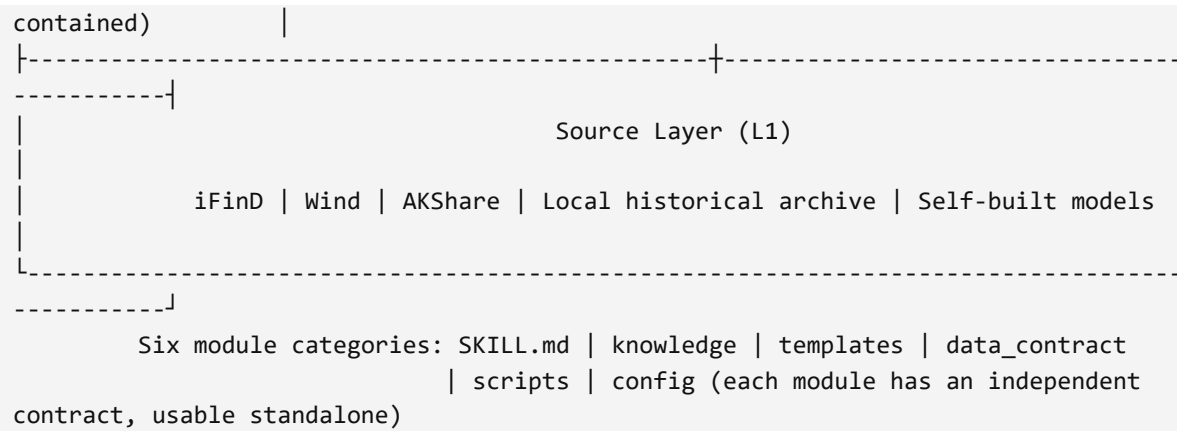
- **Module-level version management:** upgrading or rolling back a single perspective no longer disturbs the whole Skill, and version drift is greatly reduced;
- **Cross-database module reuse:** the funding-percentile module is referenced by several databases; one maintenance benefits many;
- **Manuals and code sharing the same origin:** the architecture manual is the documentation surface of a Skill 2.0 instance and can be updated in lockstep with the evolution of the code;
- **Continuous architecture evolution:** onboarding a new data source changes only the data-access layer, a new output form only extends the output layer, and the middleware framework itself remains stable.

6. Conclusion

The shift from Skill 1.0 to Skill 2.0 turns "a whole Skill is available" into "a module is available on demand." Without abandoning the position that "all research capabilities can be engineered," the modular middleware pushes encapsulation granularity down to modules, encodes contracts into IN/OUT/DEP, isolates data-source bindings into the data_contract, and hardens numeric health checks into the knowledge-layer validation baselines. The operation of 13 thematic databases and roughly 500 analytical logics in the production system provides an empirical starting point for this design; its generality will depend on whether module contracts and the validation mechanism can be established just as stably in new domains such as equity, macro, and quantitative research.

Appendix: Panoramic Architecture of the Skill 2.0 Modular Middleware





Continuously updated. Latest version: github.com/timyefi/skill20-arch